

Gestion des exceptions

1. Définitions

Une exception = condition **anormale** survenant lors de **l'exécution** d'un **élément logiciel** (bloc dans un corps de sous-programme).

- **Anormal** : $\neq$ spécifications d'utilisation normale
- **Exécution** : Si non préventif alors curatif (le compilateur ne peut pas vérifier)
- **Élément logiciel** : conséquence sur les autres éléments (propagation de l'exception)

Remarque 1 :

Dans les langages à gestion d'exception (Ada, C++, Java, Python, C#...), une exception est une donnée d'un type d'exception (objet instance d'une classe d'exception en POO).

Lorsqu'une exception survient dans un bloc alors le flot normal d'exécution est interrompu afin de gérer la situation exceptionnelle :

1. la donnée de type exception est créée afin de représenter les caractéristiques exceptionnelle
2. si un bloc de traitement adapté existe au niveau du bloc (`try {...} catch` en java), alors l'exception est transmise à ce bloc de traitement
- 2'. si aucun traitement adapté n'a été défini au niveau du bloc alors l'exception est transmise au bloc englobant ou appelant : c'est le mécanisme de propagation d'exception ; le bloc englobant ou appelant (sortie en exception d'un sous-programme) se retrouve alors lui-même en situation exceptionnelle et peut alors traiter l'exception ou la repropager à son tour...

Remarque 2 :

Si aucun bloc ne traite l'exception alors elle se propage jusqu'au point d'entrée du thread (processus léger) où a été exécuté le lancement de l'exception) et celui-ci est tué. S'il s'agit du thread principal (« main ») d'un processus alors le processus meurt en situation d'exception (provoquant une belle trace système qui fait bien paniquer les utilisateurs !).

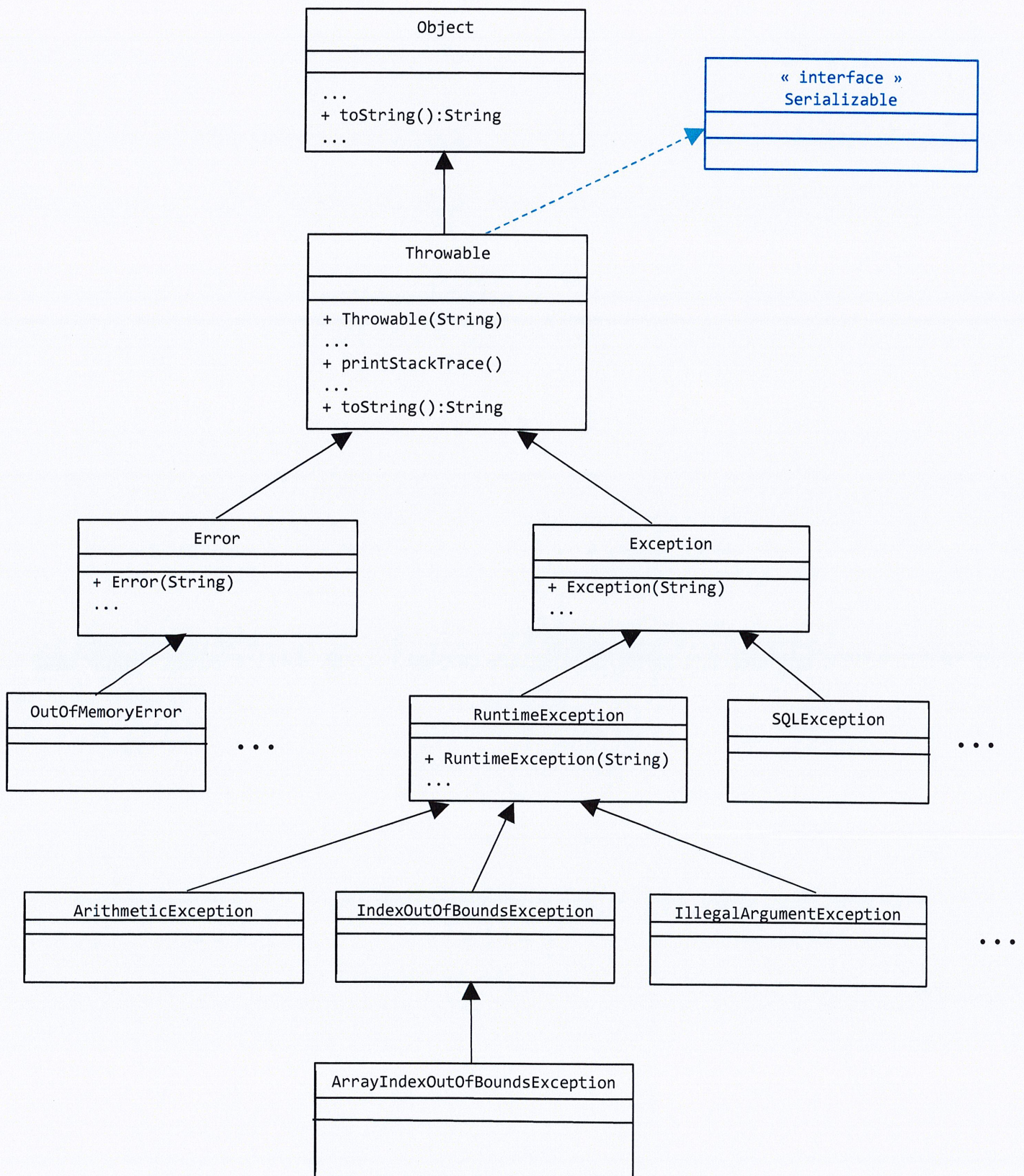
Nous allons voir que ces mécanismes sont à la base d'une bonne gestion des situations anormales en permettant des politiques adaptatives de gestion (des plus simples au plus sophistiquées) :

2. Les classes d'exception en Java

En POO, la situation anormale donne lieu à l'instanciation d'un objet d'une classe d'exception adaptée à la cause du problème. Cet objet est dit « lancé » (to throw) ou « levé » (to raise) dans le bloc.

Les classes des objets « lançables » forment une hiérarchie dans le jdk.

Sur la page 2, est présenté le sous-graphe d'héritage (en UML) des principales classes pour les exceptions et erreurs en Java = hiérarchie des classes Throwable (avec interface **Serializable**)



Voici les grandes caractéristiques des classes de base de cette hiérarchie :

Throwable : c'est la superclasse de base de la hiérarchie, elle implante les constructeurs et méthodes de base qui serviront dans les sous-classes.

Error : An Error is a subclass of Throwable that indicates serious problems that a reasonable application should not try to catch. A method is not required to declare in its throws clause any subclasses of Error that might be thrown during the execution of the method but not caught, since these errors are abnormal conditions that should never occur. That is, Error and its subclasses are regarded as unchecked exceptions for the purposes of compile-time checking of exceptions.

- ⇒ La plupart des erreurs sont dues à des problèmes « système » et ne sont pas possibles à prévenir car même déceler la condition anormale dans un test n'empêche pas que le problème puisse se produire quelques nanosecondes plus tard !
- ⇒ Le seul traitement éventuellement possible est donc de les capturer (cf. §6).

Exception : Il s'agit de la classe fondamentale des objets indiquant une situation anormale. Normalement les algorithmes devrait pouvoir traiter ces cas exceptionnels. Le mieux est un traitement préventif lorsque c'est possible (évite la situation exceptionnelle) ; à défaut l'algorithme capture l'exception pour essayer de la traiter soit complètement soit partiellement. Si un traitement complet (⇔ retour à une situation normale) n'est pas possible alors une exception est repropagée au bloc supérieur ou appelant.

Les sous-programmes doivent gérer explicitement les exceptions soit en les capturant (**try... catch**), soit en les propageant (clause **throws** dans l'entête). Hormis pour la sous-hiérarchie de RuntimeException, toute exception propagée doit être explicitement déclarée par un **throws** dans l'entête.

RuntimeException : la sous-hiérarchie de RuntimeException est particulière dans la mesure où un sous-programme susceptible de propager des exceptions de ces sous-classes n'est pas obligé de déclarer cette propagation (**throws**) explicitement dans l'entête (par contre il est conseillé de la documenter cf § 5.).

Dans la sous-hiérarchie de RuntimeException se trouvent notamment toutes les classes d'exceptions levées automatiquement par la JVM sur des situations où une prévention est possible. Ces classes permettent donc d'éviter une déclaration de propagation qui serait sans cela présente sur la plupart des sous-programmes (exemple tout algorithme effectuant des calculs en arithmétique entière est susceptible de lever une ArithmeticException...).

3. Création de classe d'exception spécifique

Lors de la conception de composants logiciels, si le développeur ne trouve pas de classe d'exception adaptée à sa situation, alors il peut déclarer lui-même une sous-classe adaptée à son contexte :

```
/**
 * « user exception » propagée en cas d'échec (« failure ») de test
 */
public static class TestException extends RuntimeException {
    /**
     * Redéfinition du constructeur sans paramètre
     * (qui n'est plus généré par défaut)
     */
    public TestException() {
        super();
    }

    /**
     * Associer un message à la cause de l'objet TestException
     * @param message texte décrivant la cause de l'exception
     */
    public TestException(String message) {
        super(message);
    }
}
```

En java, il suffit simplement de redéfinir certains constructeurs en adaptant ceux hérités de la superclasse. Les méthodes sont héritées.

Ces classes d'exception sont dites « user exceptions ».

4. Lancement d'une exception

Une exception peut être lancée de deux manières différentes :

- Automatiquement par la JVM lorsqu'une opération de niveau byte code échoue :

```
long[] enorme = new long[2000000000]; // OutOfMemoryError !

int quotient = 10 / 0; // ArithmeticException !
```

Ces erreurs et exceptions sont toujours instances de classes prédéfinies dans le jdk.

- Explicitement dans le code source à l'aide de l'instruction **throw** :

```
if (nom == null || nom.isBlank()) {
    throw new IllegalArgumentException("Patronyme inconnu");
}
```

C'est toujours le cas pour les exceptions des classes spécifiquement développées :

```
throw new TestException("combinaison invalide mais date créée !!!");
```

Une fois lancée, l'exception se propage dans le bloc où elle a lieu.

Si elle n'est pas capturée (**catch**) dans ce bloc alors elle se propage au bloc supérieur ou appelant...

5. Propagation d'une exception

La gestion de la propagation se fait donc au niveau d'un bloc.

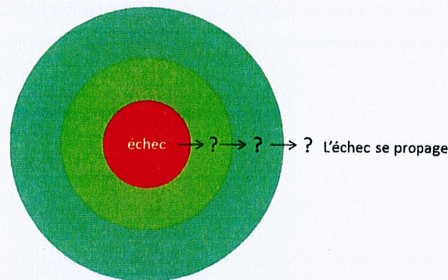
Plusieurs cas de figure se présentent alors :

① Pas de capture de cette exception

=> elle se propage au bloc supérieur ou appelant

-> assimilés à un échec du bloc = indicateur de l'échec du bloc

=> le bloc supérieur ou appelant est lui-même en exception...



=> Il faut impérativement commenter les exceptions propagées par un sous-programme dans l'interface de celui-ci.

Si l'exception n'est pas une sous-classe de `RuntimeException` alors sa propagation possible doit faire l'objet d'une clause **throws** dans l'entête du sous-programme :

```
public PrintStream(String fileName) throws FileNotFoundException {
    this(false, new FileOutputStream(fileName));
}
```

Il est impératif alors d'ajouter une explication correspondante (**@throws**) dans la documentation de l'interface :

```
/**
 * Creates a new print stream, without automatic line flushing, with the
 * specified file name. This convenience constructor creates the necessary
 * intermediate {@link java.io.OutputStreamWriter OutputStreamWriter},
 * which will encode characters using the {@linkplain Charset#defaultCharset()}
 * for this instance of the Java virtual machine.
 *
 * @param fileName
 *     The name of the file to use as the destination of this print stream.
 *     If the file exists, then it will be truncated to zero size; otherwise,
 *     a new file will be created.
 *
 * @throws FileNotFoundException
 *     If the given file object does not denote an existing, writable
 *     regular file and a new regular file of that name cannot be
 *     created, or if some other error occurs while opening or
 *     creating the file
 */
public PrintStream(String fileName) throws FileNotFoundException {
```

Si l'exception est une sous-classe de `RuntimeException` alors la clause **throws** dans l'entête est facultative ; elle est donc souvent omise.

Par contre, il est alors conseillé de commenter cette possibilité dans la documentation (**@throws**) :

```
/**
 * Date jour/mois/an si elle appartient au calendrier
 * @param jour numéro de jour dans le mois (de 1 à 31)
 * @param mois numéro de mois dans l'année (de 1 à 12)
 * @param an numéro de l'année siècle compris (de AN_MIN à AN_MAX)
 * @throws IllegalArgumentException si combinaison jour, mois, an incorrecte
 */
public Date(int jour, int mois, int an) {
```

Remarque 3 :

Le seul cas où l'on se permet d'omettre toute remarque, c'est la propagation de `NullPointerException` lorsque le sous-programme prend dans ses paramètres des références sur des objets.

En effet, si un objet utilisable par l'algorithme du corps n'est pas référencé alors cet algorithme risque d'échouer de fait puisqu'un service demandé via cette référence nulle va propager `NullPointerException`. (A moins que l'algorithme n'ait prévu de tester ce cas...)

```
/**
 * Assigns the specified int value to each element of the specified array
 * of ints.
 * @param a the array to be filled
 * @param val the value to be stored in all elements of the array
 */
public static void fill(int[] a, int val) {
    for (int i = 0, len = a.length; i < len; i++) {
        a[i] = val;
    }
}
```

⇒ Si la référence `a` est nulle, alors `a.length` lèvera `NullPointerException`

⇒ Cela n'est pas documenté car il est évident que l'on ne peut pas remplir un objet tableau s'il n'est pas référencé !

- ② On capture (**catch**) l'exception levée/lancée (**raise**, **throw**) dans le bloc et on code le traitement pour arrêter la propagation : échec du bloc -> réparation de l'échec -> assimilé à un succès
- ③ Capture de cette exception dans le bloc mais traitement partiel (pas de « nettoyage » complet du contexte) puis relance de cette exception ou d'une autre exception pour le bloc supérieur ou appelant.
 - ⇒ Signale un échec
 - ⇒ Propagation vers l'appelant pour terminer la gestion de cet échec

6. Capture et traitement d'une exception

La capture d'une exception s'effectue au niveau d'un bloc = le premier bloc dans lequel le contexte permet de traiter (complètement ou partiellement) l'exception.

```
try {
    instruction1;
    instruction2; // Lance une ExceptionA ?
    instruction3;
    instruction4; // Lance une ExceptionB ?
    instruction5; // Lance une ExceptionC ?
                // => sortie en situation d'exception c
                //    = échec (car non capturée)
    instruction6;
    // Pas d'exception => Sortie normale (succès)
} catch (ExceptionA a) {
    // Résolution de a -> remplace les instructions 2 à 6
    // => Sortie normale (pas d'échec)
} catch (ExceptionB b) {
    // Résolution partielle de b
    . . . . .
    // Relance de B => sortie en situation d'exception b = échec
    throw b ;
}
```

Syntaxe complète en java :

```
try {
    bloc surveillé
} catch (ExceptionA echecA) {
    traitement si une instruction a levé une exception
    de classe ExceptionA, echecA est alors la référence de cette
    exception
} catch (ExceptionB | ExceptionC echecBouC) {
    traitement si une instruction a levé une exception
    de classe ExceptionB ou une exception de classe ExceptionC,
    echecBouC est alors la référence de cette exception
    Attention : ExceptionB ou ExceptionC ne doivent pas être des
    spécialisations de ExceptionA
} catch (Exception autre) {
    traitement si une instruction du bloc surveillé a lancé une
    exception quelconque
    Attention : à utiliser avec parcimonie et uniquement en dernier
    recours...
} finally { // optionnel !
    traitement exécuté dans tous les cas !
}
```

Fonctionnement :

- ⇒ Si aucune exception n'est lancée alors le bloc surveillé s'exécute suivi du bloc **finally** et la situation après le bloc **finally** est considérée comme normale.
- ⇒ Si une exception est lancée, alors le premier bloc **catch** correspondant à la classe de l'exception est exécuté puis le bloc **finally** est exécuté.
Si un tel bloc **catch** correspondant est exécuté et ne relance aucune exception alors la situation après le bloc **finally** est considérée comme normale.
Si aucun bloc **catch** ne correspond au type du « throwable » alors le bloc **finally** est exécuté mais l'exception est propagée ensuite au bloc supérieur ou appelant. L'algorithme reste en situation anormale.

Exercice : quelle sera la sortie produite avec différentes valeurs des variables ?

```
int dividiende,
    diviseur,
    quotient;

/* ... */

try {
    quotient = dividiende / diviseur ;
    // ArithmeticException si division par zéro
    System.out.println("division ok");
} catch (ArithmeticException echec) {
    System.out.println("Erreur : division par zéro !");
    quotient = Integer.MAX_VALUE; // valeur par défaut
} finally {
    System.out.println("quotient = " + quotient);
}
```

Remarque 4 :

La taille du bloc surveillé doit être suffisamment restreinte pour que l'identification de l'instruction en échec soit simple.

=> Sinon, découper en plusieurs blocs surveillés juxtaposés pour limiter les possibilités d'avoir plusieurs points de lancement d'une même exception.

Remarque 5 :

Si un bloc **catch** a été exécuté alors le traitement de l'exception est considéré correct : la situation est redevenue « normale ».

=> Si l'on n'a pas pu traiter complètement l'échec alors il faut relancer une exception dans le bloc **catch** afin que la situation d'échec continue à se propager au bloc supérieur ou appelant provoquant ainsi un traitement sur plusieurs niveaux... (cf. § 7).

Remarque 6 :

Etant donné que le type de l'exception est testé en « Zorder » dans les clauses « **catch** », il faut impérativement ordonner ces « **catch** » des sous-classes les plus spécialisées vers les classes d'exception plus générales.

Sinon le traitement d'exception le plus adapté ne sera pas sélectionné au dépend d'un traitement plus général.

```
try {
    new Date(31, 2, 2020);
} catch (Exception generale) {
    System.out.println("Exception générale : " + generale.getMessage());
} catch (IllegalArgumentException neSeraPasAttrapee) {
    /* branche morte */
    System.out.println("Date invalide : "
        + neSeraPasAttrapee.getMessage());
}
```

Remarque 6' :

En langage Java, le compilateur indique cette erreur potentielle !

7. Relance d'une exception après capture

Si un traitement d'exception est considéré comme incomplet après un bloc **catch** alors ce bloc **catch** peut relancer une exception (la même ou une nouvelle).

```
try {
    bloc surveillé
} catch (ExceptionA echecA) {
    traitement partiel
    throw echecA ; // relance => situation d'échec après finally
} catch (ExceptionB | ExceptionC echecBouC) {
    traitement partiel
    // relance => situation d'échec après finally
    throw new ExceptionD(...) ;
} catch (Exception autre) {
    traitement si une instruction du bloc surveillé a lancé une
    exception quelconque.
    Ne traite pas une exception lancée dans un catch précédent !
    Attention : à utiliser avec parcimonie et uniquement en dernier
    recours...
} finally { // optionnel !
    traitement exécuté dans tous les cas !
}
```

Remarque 7 :

Une exception lancée par une instruction d'un bloc **catch** ne sera pas traitée par un **catch** relatif au même bloc surveillé mais propagée au bloc supérieur ou appelant.

=> Dès qu'un bloc **catch** a été exécuté alors son exécution est exclusive dans la structure de contrôle : aucun autre bloc **catch** de la même structure ne sera exécuté.

=> Cela permet une politique de gestion adaptative sur plusieurs niveaux d'une situation d'échec.